

CineNode — Replit Integration Plan

From the current GitHub repository to a live **.replit.app** deployment, with Replit Agent meaningfully in the loop.

The requirement, verbatim

“Your project must be built using Replit Agent as part of the development process, and the finished project must be hosted and deployed directly on Replit (a project URL on a **replit.app** or **replit.dev** domain). Projects not deployed on Replit’s platform will not meet this requirement, regardless of how the code was written.”

Two things must be true, and they are separate: Agent must have **done real work**, and the app must be **deployed on Replit**. This plan satisfies both without rewriting the project.

1. Where the repository stands

Area	Current state
Frontend	React 18.3 + Vite 5.4 + react-router-dom 7, vanilla-CSS design tokens. Builds to frontend/dist .
Backend	FastAPI (backend/main.py), routers in domains/ , services in services/ . API-only — serves no static files.
FE ↔ BE	Frontend calls relative /api/... paths only — no hardcoded backend URL anywhere. Dev uses a Vite proxy. This is why single-origin serving needs no frontend changes.
Auth	PBKDF2-SHA256 + opaque bearer sessions, per-production membership enforced on every route. Dual-mode store.
AI	services/gemini_client.py → gemini-3.6-flash with a fallback chain. 14 calls per pipeline run, 9 per audience simulation.
Run locally	make backend (uvicorn :8000) + make frontend (vite :5173) — two processes.
Deployed today	Two separate Cloud Run services (backend Dockerfile, frontend nginx Dockerfile).

2. What blocked Replit — and what is already fixed

The first three are now resolved in the repository. The last three are the Agent’s work.

#	Blocker	Status
1	.gitignore corruption. Last line was UTF-16LE .env appended to a UTF-8 file, so git treated it as binary.	FIXED Claude Code

#	Blocker	Status
2	Health path mismatch. Dockerfile probed <code>/health</code> ; the app serves <code>/api/health</code> .	FIXED Claude Code
3	Dead config. WHISPER / IMAGEN / CLOUD_SQL keys and <code>cloudsql_client.py</code> were never imported; docs falsely claimed Cloud SQL worked.	FIXED Claude Code
4	Two processes, two ports. Replit exposes one HTTP port. FastAPI must serve the built SPA and <code>/api</code> together, on <code>0.0.0.0:\$PORT</code> .	TO DO Replit Agent
5	Ephemeral filesystem. <code>backend/.state/</code> holds users, sessions, invites, <code>GlobalState</code> and simulations. On Replit it is wiped on redeploy — accounts would silently vanish.	TO DO Replit Agent
6	Background jobs vs recycling. A simulation runs on a thread for ~4 minutes after returning 202. A recycled instance leaves it stuck at “running” forever.	TO DO Replit Agent

Already done — no action needed

Claude Code has fixed blockers 1–3, activated **supabase** in `requirements.txt`, added a **CINENODE_STATE_BACKEND** override (`auto / supabase / local`) so the Supabase switch can be forced or disabled, and added **REPLIT.md** — an architecture brief that Agent reads on import. Verified: backend imports (35 routes), full mock pipeline runs (132 A2A messages), frontend builds.

3. Who does what, and why

The split is deliberate. Everything left for Agent is genuinely Replit-native work that produces visible artifacts — not busywork staged to look like contribution.

Claude Code — before the move (done)	Replit Agent — after import
Repo hygiene and correctness: encoding fix, health path, dead-code removal, dependency pinning, the state-backend override, and the REPLIT.md brief.	Single-port serving and .replit deployment config; activating Supabase persistence; making long simulations survive instance restarts; deploying.
Cheap, precise, and would have cost many Agent iterations to rediscover.	Replit-native problems Agent is well suited to, each landing in the product.

Deliberate choice

Single-port serving was **not** pre-built in Claude Code. It is the most Replit-specific change in the project, so leaving it to Agent is what makes Agent's involvement substantive rather than cosmetic.

4. Step by step: GitHub → Replit → deployment

Step 1 — Import the repository

Import github.com/Shriya-Soni/CineNode into Replit via *Import from GitHub*. Import the whole repo unchanged — `node_modules`, `.venv`, `dist`, `.state` and `.env` are all gitignored, so it is clean.

Keep the GitHub link active so Agent's commits push back to the repo. This preserves your existing workflow *and* doubles as Agent evidence.

Step 2 — Add Secrets before running anything

Use Replit's **Secrets** pane. They arrive as environment variables, which `backend/core/config.py` already reads via `os.environ.get`.

Secret	Purpose	Required
GEMINI_API_KEY	All agent reasoning	For live AI
SUPABASE_URL	Shared persistence endpoint	Yes
SUPABASE_KEY	Shared persistence key	Yes
TAVILY_API_KEY	Optional web grounding for cultural research	No

Do not create any VITE_-prefixed secret

Vite inlines `VITE_*` variables into the public JavaScript bundle. Every key in this project is read server-side and the frontend only calls relative `/api` paths, so nothing sensitive reaches the browser. Keep it that way.

Step 3 — Run the four Agent prompts

In order. Each is scoped to keep credit use low and to stop Agent exploring the repo on your budget.

PROMPT 1 · Single-port serving + .replit config

This is an existing FastAPI + React (Vite) app. Do not rewrite it. The React frontend is in frontend/ and builds to frontend/dist; the FastAPI app is backend/main.py and serves all routes under /api. Make it run as a SINGLE web process on Replit: mount frontend/dist on FastAPI as static files, add a catch-all SPA fallback returning index.html for non-/api paths so client-side routes like /casting and /logs work on refresh, and bind 0.0.0.0 on \$PORT. Create the .replit config with a build step that installs backend/requirements.txt and runs the Vite build. Do not change any API route paths or the frontend's relative /api fetch calls.

PROMPT 2 · Activate Supabase persistence

Data currently persists to JSON files under backend/.state/, which won't survive Replit deployments. services/supabase_client.py, services/auth_store.py and services/simulation_store.py already have a Supabase branch that activates when SUPABASE_URL / SUPABASE_KEY are set and the supabase package is installed. Apply the schema in backend/schema_auth.sql to our Supabase project, add the equivalent tables for global_state and cn_simulations, and verify register, login, invite and run-pipeline all persist across a restart. Keep the local-JSON fallback working for offline dev.

PROMPT 3 · Make simulations restart-safe

Audience simulations in backend/domains/audience/router.py run on a background thread for several minutes. If the instance restarts mid-run the record is stuck at status "running" forever. Make this resilient: on startup, mark any simulation still running as failed with a clear message, and surface that in the Phase V dashboard so the user can re-run. Do not change the simulation algorithm.

PROMPT 4 · Deploy and hand back the URL

Deploy this as a Reserved VM deployment. Confirm the build runs the Vite build before starting uvicorn, that secrets GEMINI_API_KEY, SUPABASE_URL and SUPABASE_KEY are read from Replit Secrets, and give me the public .replit.app URL.

5. Deployment configuration

Choose Reserved VM.

Autoscale is cheaper, but it scales to zero — which kills a four-minute background simulation mid-run and puts a cold start in front of the judges. Reserved VM stays warm and lets background threads finish.

```
# build
npm --prefix frontend ci && npm --prefix frontend run build
pip install -r backend/requirements.txt

# run (one process, one port)
uvicorn main:app --app-dir backend --host 0.0.0.0 --port $PORT
```

6. Verifying the requirement is met

The submission URL must be a published **deployment** on **.replit.app**. The workspace preview on **.replit.dev** sleeps when the workspace idles, so it is the weaker choice even though the rules allow it. Run these checks in a private window:

- ☐ `https://<name>.replit.app/api/health` returns `{"status":"ok"}`
- ☐ Hard-refresh `/casting` directly — it loads rather than 404ing (proves the SPA fallback)
- ☐ Register a new account, sign in, run the pipeline, run an audience simulation
- ☐ Redeploy, then confirm the same account still signs in (proves Supabase, not ephemeral JSON)
- ☐ View source and the network tab — **no API keys anywhere in the bundle**
- ☐ Open the URL on a phone — the layout is responsive down to 375px

7. Evidence to keep for Devpost

The requirement is about process, not just hosting, so the evidence matters as much as the URL.

- ☐ Screenshots of the Agent chat for each of the four prompts — before and after
- ☐ Replit's **checkpoint history** for the session
- ☐ **git log** showing Replit-authored commits alongside Claude Code's — the strongest single artifact
- ☐ The **.replit** file, which Agent generates
- ☐ The deployment log and the live **.replit.app** URL
- ☐ A short screen recording of the deployed app running a real simulation

Demo risk worth planning around

A live audience simulation is ~9 Gemini calls at roughly 20 seconds each — several minutes with nothing to look at. Run one **before** presenting and show the stored result from the history table, which also demonstrates the run-comparison feature. Keep a live run as the encore, not the opening.

Prepared by Claude Code · blockers 1–3 already fixed in the repository · repo: github.com/Shriya-Soni/CineNode